

CUDA and Parallel Computing: Basic Terminology Notes

1 Processor and Performance Terms

1.1 Clock Frequency

Clock frequency is the number of processor clock cycles per second, measured in Hertz (Hz). For a frequency f , the clock period is

$$T = \frac{1}{f}.$$

For example, 3 GHz means 3×10^9 clock cycles per second.

A higher frequency does *not* automatically mean higher application performance. Performance also depends on how much useful work is completed per cycle, memory stalls, instruction dependencies, branch behavior, and available parallelism.

A useful distinction is:

Term	Meaning	Typical unit
Frequency	Clock cycles per second	Hz, GHz
Bandwidth	Data transferred per second	GB/s
Latency	Time required for one operation/access	ns, cycles

Instruction throughput is often approximated by

$$\text{Instruction throughput} \approx \text{Clock frequency} \times \text{IPC},$$

where IPC means **Instructions Per Cycle**.

1.2 ALU and FPU

An **Arithmetic Logic Unit (ALU)** is a hardware execution unit that performs integer arithmetic and logical operations such as addition, subtraction, comparison, shifts, AND, OR, and XOR.

A **Floating-Point Unit (FPU)** performs floating-point arithmetic. Modern processors may contain several types of execution units, including integer ALUs, floating-point units, vector units, and specialized matrix units.

The key idea is:

Execution units are the hardware that actually performs computation.

1.3 Control Logic

Control logic manages how instructions move through a processor. Its responsibilities include instruction fetch and decode, dependency handling, scheduling, branch handling, register access, and load/store control.

Important related terms:

- **Program Counter (PC):** stores the address of the next instruction.
- **Instruction fetch:** retrieves the next instruction.
- **Instruction decode:** determines what operation the instruction requests.
- **Branch prediction:** predicts the outcome of a branch so execution can continue before the result is known.
- **Out-of-order execution:** allows independent instructions to execute before earlier stalled instructions, while preserving the correct final program result.

CPUs devote substantial hardware to sophisticated control logic because they are optimized for low-latency execution of a relatively small number of complex instruction streams. GPUs devote proportionally more hardware to arithmetic throughput and parallel execution.

2 Memory Terms

2.1 Cache

A **cache** is a small, fast memory close to the processor that stores recently or frequently used data and instructions. Its purpose is to reduce the average cost of accessing slower main memory.

A simplified memory hierarchy is

registers $\rightarrow$ cache $\rightarrow$ DRAM $\rightarrow$ storage.

Two locality principles explain why caches work:

- **Temporal locality:** recently accessed data is likely to be accessed again soon.
- **Spatial locality:** data near a recently accessed address is likely to be accessed soon.

A **cache hit** means the requested data is already in the cache. A **cache miss** means the processor must access a lower and slower memory level.

Modern CPUs commonly use multiple cache levels:

Level	Relative speed	Relative size	Typical sharing
L1	fastest	smallest	usually per core
L2	fast	larger	per core or core group
L3	slower	much larger	shared by several cores
DRAM	much slower	far larger	system main memory

Caches are typically implemented using **SRAM**, which is fast but consumes more chip area per bit than DRAM.

2.2 Cache Line

A **cache line** is the fixed-size block transferred between cache levels or between cache and memory. A common CPU cache-line size is 64 bytes.

This is why sequential memory access usually benefits from spatial locality: reading one value often brings nearby values into the cache at the same time.

2.3 DRAM

Dynamic Random-Access Memory (DRAM) is the main memory used to store program data while the computer is running. Large arrays, heap objects, program code, and operating-system data ultimately reside in DRAM unless currently cached closer to the processor.

DRAM provides large capacity at relatively low cost per bit, but its access latency is much higher than cache latency.

DRAM performance is commonly discussed using two different quantities:

- **Latency:** how long one memory access takes.
- **Bandwidth:** how much data can be transferred per unit time.

GPUs use high-bandwidth memory systems because thousands of active threads may request data simultaneously.

3 CUDA Execution Model

3.1 Kernel

A **kernel** is a function that runs on the GPU and is executed by many CUDA threads in parallel.

A kernel launch defines how many threads will execute by specifying a **grid** of **thread blocks**.

3.2 Thread, Block, and Grid

A **thread** is the smallest logical execution unit exposed by CUDA.

A **thread block** is a group of threads that can cooperate using shared memory and block-level synchronization.

A **grid** is the complete collection of blocks created by one kernel launch.

The hierarchy is

kernel launch $\rightarrow$ grid $\rightarrow$ blocks $\rightarrow$ threads.

For a one-dimensional grid, a common global thread index is

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}.$$

This expression maps a thread's position in the grid to a data index.

3.3 Warp

A **warp** is the basic scheduling and execution group on NVIDIA GPUs. A warp contains 32 threads.

CUDA exposes individual threads, but the hardware schedules them in warps. Threads in one warp normally execute the same instruction on different data. NVIDIA calls this execution model **Single Instruction, Multiple Threads (SIMT)**.

3.4 Warp Divergence

Warp divergence occurs when threads in the same warp follow different control-flow paths. The warp may then need to execute the paths separately while disabling threads that are not active on the current path.

Therefore, divergence can reduce effective parallelism.

3.5 Shared Memory

Shared memory is a low-latency on-chip memory shared by threads in the same block. It is explicitly managed by CUDA programs and is commonly used for data reuse, tiling, and communication among threads in one block.

3.6 Memory Bank and Bank Conflict

Shared memory is divided into **memory banks** so multiple accesses can be serviced in parallel.

A **bank conflict** occurs when multiple threads in the same warp access different addresses that map to the same bank. These accesses may need to be serialized.

For a simplified 32-bank model with 32-bit words,

$$\text{bank index} \approx \text{word index} \bmod 32.$$

3.7 Global Memory

Global memory is the large off-chip memory visible to GPU threads. It has much larger capacity than shared memory or registers but also much higher access latency.

Efficient CUDA kernels therefore try to use global memory bandwidth effectively and avoid unnecessary memory traffic.

3.8 Memory Coalescing

Memory coalescing means organizing global-memory accesses so neighboring threads in a warp access nearby memory addresses that can be served by a small number of memory transactions.

Coalesced access improves effective memory bandwidth.

3.9 Synchronization

Synchronization coordinates threads that depend on each other's work.

Within a thread block,

```
__syncthreads();
```

acts as a barrier: threads in the block wait until all participating threads have reached the synchronization point before continuing.

4 CPU and GPU Design

A CPU is primarily **latency-oriented**: it uses sophisticated control logic, large caches, branch prediction, and out-of-order execution to make a relatively small number of instruction streams complete quickly.

A GPU is primarily **throughput-oriented**: it uses many execution units and massive thread-level parallelism to complete a large amount of work per unit time.

The central distinction is

CPU: optimize the latency of a few complex threads

GPU: optimize the throughput of many parallel threads

CUDA allows one program to use both: the CPU acts as the **host**, while the GPU acts as the **device**.

5 Parallel Programming Interfaces

5.1 CUDA

Compute Unified Device Architecture (CUDA) is NVIDIA's programming platform and execution model for general-purpose computing on NVIDIA GPUs.

CUDA exposes GPU concepts such as kernels, grids, blocks, threads, synchronization, and the GPU memory hierarchy.

5.2 OpenMP

Open Multi-Processing (OpenMP) is a directive-based programming interface mainly used for shared-memory parallelism. It is commonly used to parallelize loops across CPU cores and can also target some accelerators.

Typical abstraction:

OpenMP $\approx$ shared-memory thread parallelism

5.3 MPI

Message Passing Interface (MPI) is the standard programming interface for distributed-memory parallel computing. Different processes or compute nodes own separate memory and exchange data explicitly through messages.

Typical abstraction:

 $\text{MPI} \approx \text{parallelism across processes or compute nodes}$

5.4 NCCL

NVIDIA Collective Communications Library (NCCL) is an NVIDIA library optimized for communication among GPUs.

It provides collective operations such as broadcast, reduce, all-reduce, all-gather, and reduce-scatter. NCCL is widely used in distributed deep learning.

Typical abstraction:

 $\text{NCCL} \approx \text{high-performance collective communication among GPUs}$

5.5 OpenCL

Open Computing Language (OpenCL) is a cross-vendor programming standard for heterogeneous computing on CPUs, GPUs, and other accelerators.

Its execution model is conceptually similar to CUDA, but its terminology differs:

CUDA	OpenCL	Meaning
Thread	Work-item	Logical execution unit
Thread block	Work-group	Cooperative group of threads
Grid	NDRange	Full execution space
Shared memory	Local memory	Memory shared within a group
Global memory	Global memory	Device-wide memory

5.6 Quick Comparison

Interface	Main purpose	Typical scope
CUDA	Explicit GPU programming	NVIDIA GPU
OpenMP	Shared-memory parallelism	Multicore CPU / accelerator
MPI	Distributed-memory parallelism	Multiple processes / nodes
NCCL	GPU collective communication	Multiple NVIDIA GPUs
OpenCL	Portable accelerator programming	Cross-vendor devices

6 Quick Lookup Summary

Term	Short definition
Clock frequency	Number of processor clock cycles per second.
Latency	Time required to complete one operation or access.
Bandwidth	Amount of data transferred per unit time.
ALU	Hardware unit for integer arithmetic and logic.
FPU	Hardware unit for floating-point arithmetic.
Control logic	Hardware that fetches, decodes, schedules, and coordinates instructions.
Cache	Small fast memory used to reduce average memory-access latency.
Cache line	Fixed-size block moved through the cache hierarchy.
DRAM	Large-capacity main memory with relatively high latency.
Kernel	Function executed on the GPU by many threads.
Thread	Smallest logical CUDA execution unit.
Block	Cooperative group of CUDA threads.
Grid	All blocks created by one kernel launch.
Warp	Hardware execution group of 32 NVIDIA GPU threads.
SIMT	One instruction applied to multiple GPU threads with independent thread state.
Warp divergence	Threads in one warp following different control-flow paths.
Shared memory	Fast on-chip memory shared by threads in one block.
Global memory	Large off-chip GPU memory visible to all threads.
Memory bank	Parallel partition of shared memory.
Bank conflict	Multiple warp threads contending for the same shared-memory bank.
Memory coalescing	Combining nearby global-memory accesses from a warp into efficient transactions.
Synchronization	Coordination between threads when execution has dependencies.
OpenMP	Shared-memory parallel programming interface.
MPI	Distributed-memory message-passing interface.
NCCL	Collective communication library optimized for NVIDIA GPUs.
OpenCL	Cross-vendor heterogeneous parallel programming standard.
CUDA	NVIDIA platform and programming model for GPU computing.